

AWS S3 Design — b-secure Platform

Production S3 design reference: bucket structure, IAM, pre-signed URLs, lifecycle rules, CloudFront, and cost optimisation.

Table of Contents

- 1. [Overview](#)
- 2. [Bucket Structure](#)
- 3. [Django Storage Configuration](#)
- 4. [IAM Policy \(Least Privilege\)](#)
- 5. [Pre-Signed URLs](#)
- 6. [Access Patterns](#)
- 7. [CORS Configuration](#)
- 8. [Lifecycle Rules](#)
- 9. [Versioning & Backup](#)
- 10. [Security](#)
- 11. [CloudFront Distribution](#)
- 12. [Cost Optimisation](#)

1. Overview

S3 is the primary object store for all binary assets in b-secure. No files are stored on EC2/ECS instance storage.

Content Type	Bucket Prefix	Access Pattern	Sensitivity
Guard profile photos	media/guards/photos/	Public via CloudFront	Low
Guard identity documents (Aadhaar, PAN)	media/guards/documents/	Private, pre-signed URL (admin only)	High
Guard ID proof photos	media/guards/id_proof/	Private, pre-signed URL	High
			Low

Content Type	Bucket Prefix	Access Pattern	Sensitivity
User profile photos	media/users/photos/	Public via CloudFront	Medium
Booking receipt PDFs	media/bookings/receipts/	Pre-signed URL (user, 1h TTL)	
Analytics CSV exports	exports/analytics/	Pre-signed URL (admin, 15 min TTL)	Medium
Payout reports	exports/payouts/	Pre-signed URL (admin, 15 min TTL)	Medium
Static assets (JS/CSS/fonts)	static/	Public via CloudFront	None

2. Bucket Structure

2.1 One Bucket Per Environment

Environment	Bucket Name	Region
Development	bsecure-dev	ap-south-1 (Mumbai)
Staging	bsecure-staging	ap-south-1
Production	bsecure-prod	ap-south-1
DR Replica	bsecure-prod-replica	ap-south-2 (Hyderabad)

2.2 Folder Hierarchy with Key Examples

```

bsecure-prod/
├── media/
│   ├── guards/
│   │   ├── photos/
│   │   │   ├── {guard_id}/
│   │   │   │   ├── profile.jpg
│   │   │   │   # e.g. media/guards/photos/42/profile.jpg
│   │   │   │   # e.g. media/guards/photos/42/
│   │   │   profile_thumb_200x200.jpg
│   │   ├── documents/
│   │   │   ├── {guard_id}/
│   │   │   │   ├── {doc_type}/
│   │   │   │   │   ├── {filename}
│   │   │   │   # e.g. media/guards/documents/42/
│   │   │   police_clearance/cert_2025.pdf

```

```

| | | # e.g. media/guards/documents/42/
training_certificate/cert.jpg
| | |   |
| | |   |— id_proof/
| | |   |   |
| | |   |   |— {guard_id}/
| | |   |   |   |
| | |   |   |   |— {filename}
| | |   |   |   | # e.g. media/guards/id_proof/42/aadhaar_front.jpg
| | |   |   |   | # e.g. media/guards/id_proof/42/pan_card.jpg
| | |   |— users/
| | |   |   |
| | |   |   |— photos/
| | |   |   |   |
| | |   |   |   |— {user_id}/
| | |   |   |   |   |
| | |   |   |   |   |— profile.jpg
| | |   |   |   |   | # e.g. media/users/photos/101/profile.jpg
| | |   |— bookings/
| | |   |   |
| | |   |   |— receipts/
| | |   |   |   |
| | |   |   |   |— {booking_id}/
| | |   |   |   |   |
| | |   |   |   |   |— receipt.pdf
| | |   |   |   |   | # e.g. media/bookings/receipts/4891/receipt.pdf
|— exports/
|   |
|   |— analytics/
|   |   |
|   |   |— {date}/
|   |   |   |
|   |   |   |— report.csv
|   |   |   | # e.g. exports/analytics/2025-05-28/daily_bookings.csv
|   |— payouts/
|   |   |
|   |   |— {month}/
|   |   |   |
|   |   |   |— payout_report.xlsx
|   |   |   | # e.g. exports/payouts/2025-05/guard_payouts.xlsx
|— static/
|   |
|   |— admin/ # Django admin static files
|   |— assets/ # App-level static (served via
CloudFront)

```

2.3 S3 Key Construction in Django

```

# media/upload_paths.py
import os
import uuid

def guard_photo_path(instance, filename):
    ext = os.path.splitext(filename)[1].lower()
    return f"media/guards/photos/{instance.id}/profile{ext}"

def guard_document_path(instance, filename):
    ext = os.path.splitext(filename)[1].lower()
    safe_name = f"{uuid.uuid4().hex}{ext}"
    return f"media/guards/documents/{instance.guard_id}/"
    {instance.document_type}/{safe_name}"

def guard_id_proof_path(instance, filename):
    ext = os.path.splitext(filename)[1].lower()
    safe_name = f"{uuid.uuid4().hex}{ext}"

```

```

    return f"media/guards/id_proof/{instance.guard_id}/{
        {safe_name}}"

def user_photo_path(instance, filename):
    ext = os.path.splitext(filename)[1].lower()
    return f"media/users/photos/{instance.id}/profile{ext}"

def booking_receipt_path(instance, filename):
    return f"media/bookings/receipts/{instance.booking_id}/
        receipt.pdf"

```

3. Django Storage Configuration

3.1 Install Dependencies

```
pip install django-storages[boto3] boto3
```

3.2 settings/base.py

```

import os

# — AWS credentials (use IAM role on ECS; env vars for local dev)
AWS_ACCESS_KEY_ID      = os.environ.get("AWS_ACCESS_KEY_ID", "")
AWS_SECRET_ACCESS_KEY  = os.environ.get("AWS_SECRET_ACCESS_KEY",
    "")
AWS_S3_REGION_NAME     = "ap-south-1"
AWS_STORAGE_BUCKET_NAME =
    os.environ.get("AWS_STORAGE_BUCKET_NAME", "bsecure-dev")

# — CloudFront domain (CDN in front of S3)
AWS_S3_CUSTOM_DOMAIN   = os.environ.get("AWS_CLOUDFRONT_DOMAIN",
    f"{AWS_STORAGE_BUCKET_NAME}.s3.
    {AWS_S3_REGION_NAME}.amazonaws.com")

# — Default storage
DEFAULT_FILE_STORAGE    = "config.storages.MediaStorage"
STATICFILES_STORAGE     = "config.storages.StaticStorage"

MEDIA_URL   = f"https://{AWS_S3_CUSTOM_DOMAIN}/media/"
STATIC_URL  = f"https://{AWS_S3_CUSTOM_DOMAIN}/static/"

# — S3 behaviour
AWS_DEFAULT_ACL          = None          # use bucket policy,
    not ACLs

```

```

AWS_S3_FILE_OVERWRITE      = False          # preserve original on
                                     re-upload
AWS_QUERYSTRING_AUTH       = False          # public files don't
                                     need query params
AWS_S3_OBJECT_PARAMETERS   = {
    "CacheControl": "max-age=86400",        # default: 1 day
    browser cache
}
AWS_S3_SIGNATURE_VERSION   = "s3v4"
AWS_S3_ADDRESSING_STYLE    = "virtual"      #
                                     bucket.s3.amazonaws.com style

```

3.3 Custom Storage Classes

```

# config/storages.py
from storages.backends.s3boto3 import S3Boto3Storage

class MediaStorage(S3Boto3Storage):
    """Default storage for user-uploaded media (profile photos,
    etc.)."""
    location      = "media"
    default_acl    = None
    # private bucket; CloudFront OAC handles access

class StaticStorage(S3Boto3Storage):
    """Static files served via CloudFront."""
    location      = "static"
    default_acl    = None

class PrivateDocumentStorage(S3Boto3Storage):
    """
    Storage for sensitive documents (ID proofs, police
    clearances).
    Files are never publicly accessible – always accessed via pre-
    signed URLs.
    Uses SSE-KMS instead of default SSE-S3.
    """
    location      = "media"
    default_acl    = None
    object_parameters = {
        "ServerSideEncryption": "aws:kms",
        "SSEKMSKeyId":          "arn:aws:kms:ap-
south-1:123456789:key/your-kms-key-id",
    }

    def url(self, name, parameters=None, expire=300,
            http_method=None):

```

```
"""Override: always return a pre-signed URL with 5 min TTL
for private docs."""
return self._generate_presigned_url(name,
expiration=expire)
```

4. IAM Policy (Least Privilege)

4.1 Application IAM User/Role Policy

The ECS task role and the application IAM user (for local dev) should have the minimum permissions required.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowMediaReadWrite",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject",
        "s3:PutObjectTagging",
        "s3:GetObjectTagging"
      ],
      "Resource": "arn:aws:s3:::bsecure-prod/media/*"
    },
    {
      "Sid": "AllowStaticWrite",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject"
      ],
      "Resource": "arn:aws:s3:::bsecure-prod/static/*"
    },
    {
      "Sid": "AllowPresignedURLGeneration",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::bsecure-prod/media/guards/id_proof/*",
        "arn:aws:s3:::bsecure-prod/media/guards/documents/*",
        "arn:aws:s3:::bsecure-prod/media/bookings/receipts/*"
      ]
    }
  ],
}
```

```

    {
      "Sid": "DenyListBucketForApp",
      "Effect": "Deny",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::bsecure-prod"
    }
  ]
}

```

4.2 Admin IAM Role Policy (Exports + Full Media)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AdminFullMediaAccess",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::bsecure-prod",
        "arn:aws:s3:::bsecure-prod/*"
      ]
    },
    {
      "Sid": "AdminExportsAccess",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::bsecure-prod/exports/*"
      ]
    }
  ]
}

```

4.3 KMS Key Policy for Identity Documents

```

{
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::123456789:role/bsecure-ecs-task"
  },
  "Action": [

```

```

        "kms:GenerateDataKey",
        "kms:Decrypt"
    ],
    "Resource": "*",
    "Condition": {
        "StringEquals": {
            "kms:ViaService": "s3.ap-south-1.amazonaws.com",
            "kms:CallerAccount": "123456789"
        }
    }
}

```

5. Pre-Signed URLs

5.1 Why Pre-Signed URLs

- The S3 bucket is fully private (Block Public Access enabled)
- CloudFront OAC serves *public* content (profile photos, static assets)
- Sensitive documents (ID proofs, receipts) must be time-limited and user-specific
- Pre-signed URLs embed temporary AWS credentials in the URL — no server proxy required

5.2 Generate Pre-Signed GET URL

```

# core/s3_utils.py
import boto3
from botocore.exceptions import ClientError
from django.conf import settings
import logging

logger = logging.getLogger(__name__)

def get_s3_client():
    """Returns a boto3 S3 client. Uses IAM role on ECS; env vars locally."""
    return boto3.client(
        "s3",
        region_name=settings.AWS_S3_REGION_NAME,
    )

def generate_presigned_get_url(s3_key: str, expiration_seconds:
    int = 3600) -> str | None:
    """
    Generate a pre-signed GET URL for a private S3 object.

```


Args:

*s3_key: Full S3 key, e.g. "media/guards/
id_proof/42/aadhaar_front.jpg"
expiration_seconds: URL validity. Use 300 for ID docs,
3600 for receipts.*

Returns:

Pre-signed URL string or None on error.

```
"""
s3 = get_s3_client()
try:
    url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": settings.AWS_STORAGE_BUCKET_NAME,
            "Key": s3_key,
        },
        ExpiresIn=expiration_seconds,
    )
    return url
except ClientError as e:
    logger.error("Failed to generate pre-signed GET URL for  
%s: %s", s3_key, e)
    return None
```

5.3 Generate Pre-Signed PUT URL (Direct Client Upload)

```
def generate_presigned_put_url(s3_key: str, content_type: str,
    expiration_seconds: int = 300) -> dict | None:
    """
    Generate a pre-signed PUT URL so the client can upload
    directly to S3.
    Server never receives the file – reduces bandwidth and server
    load.

    Returns dict with 'url' and 'fields' (for multipart) or just
    'url' for PUT.
    """
    s3 = get_s3_client()
    try:
        # For PUT (simpler, single-part uploads up to 5 GB)
        url = s3.generate_presigned_url(
            ClientMethod="put_object",
            Params={
                "Bucket": settings.AWS_STORAGE_BUCKET_NAME,
                "Key": s3_key,
                "ContentType": content_type,
            },
            ExpiresIn=expiration_seconds,
        )
        return {"url": url, "key": s3_key, "method": "PUT"}
    except ClientError as e:
```

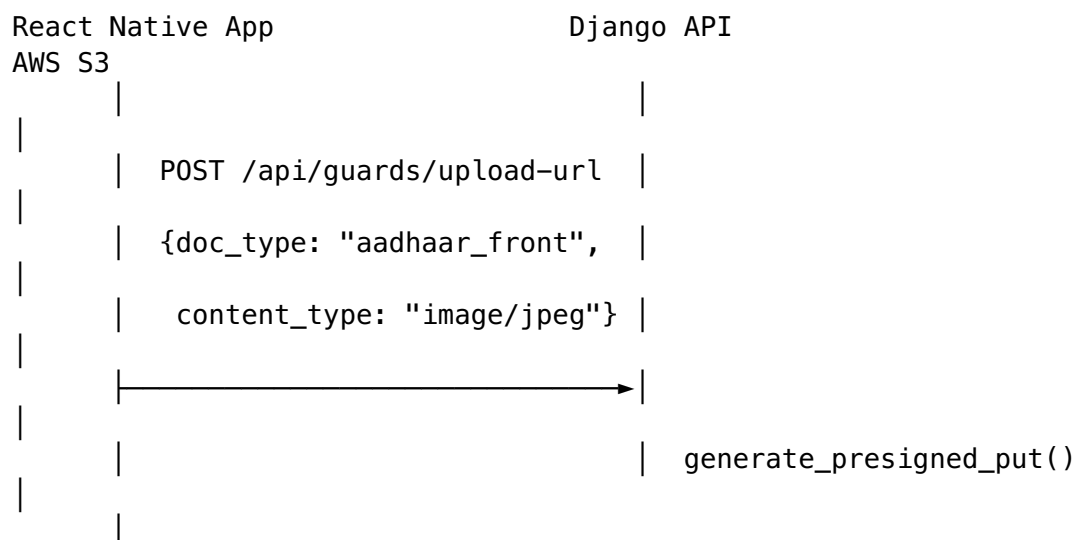
```

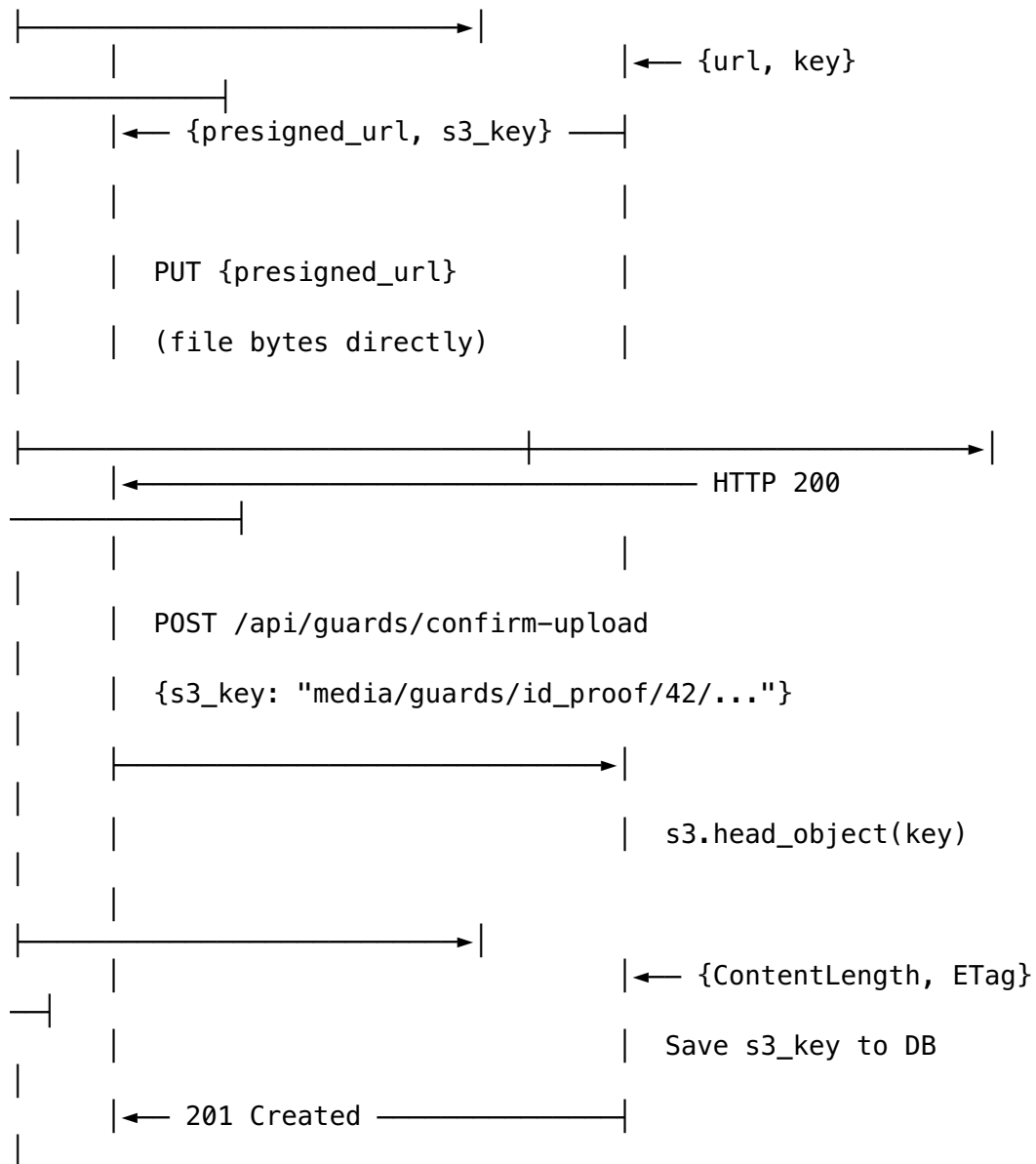
        logger.error("Failed to generate pre-signed PUT URL for
%s: %s", s3_key, e)
        return None

def generate_presigned_post(s3_key_prefix: str, content_type: str,
                           max_size_bytes: int = 10 * 1024 *
                           1024,
                           expiration_seconds: int = 300) ->
    dict | None:
    """
    Generate a pre-signed POST URL (for React Native FormData
    uploads).
    Supports conditions like max file size and content type
    enforcement.
    """
    s3 = get_s3_client()
    try:
        response = s3.generate_presigned_post(
            Bucket=settings.AWS_STORAGE_BUCKET_NAME,
            Key=s3_key_prefix,
            Fields={
                "Content-Type": content_type,
            },
            Conditions=[
                {"Content-Type": content_type},
                ["content-length-range", 1, max_size_bytes],
            ],
            ExpiresIn=expiration_seconds,
        )
        return response  # {"url": "...", "fields": {"key": ...,
        "Content-Type": ..., ...}}
    except ClientError as e:
        logger.error("Failed to generate presigned POST: %s", e)
        return None

```

5.4 Direct-to-S3 Upload Flow





guards/views.py – upload URL endpoint

```

class GuardDocumentUploadUrlView(APIView):
    permission_classes = [IsAuthenticated, IsGuard]

    def post(self, request):
        doc_type = request.data.get("doc_type") #
        # "aadhaar_front", "pan_card", etc.
        content_type = request.data.get("content_type", "image/
        jpeg")

        ALLOWED_DOC_TYPES = {"aadhaar_front", "aadhaar_back",
        "pan_card", "police_clearance", "training_cert"}
        if doc_type not in ALLOWED_DOC_TYPES:
            return Response({"detail": "Invalid doc_type."},
            status=400)

        guard = request.user.guard
        ext = "pdf" if content_type == "application/pdf"
        else "jpg"
  
```

```

s3_key = f"media/guards/documents/{guard.id}/{doc_type}/{
    uuid.uuid4().hex}.{ext}"

result = generate_presigned_put_url(
    s3_key=s3_key,
    content_type=content_type,
    expiration_seconds=300,
)
if not result:
    return Response({"detail": "Could not generate upload
URL."}, status=500)

return Response({
    "upload_url": result["url"],
    "s3_key": s3_key,
    "expires_in": 300,
})

```

6. Access Patterns

Asset	Storage Class	Access Method	TTL	Who
Guard profile photo	S3 Standard	CloudFront public URL	1 day (CDN cache)	Anyone
Guard ID / Aadhaar	S3 Standard (SSE-KMS)	Pre-signed GET URL	300s (5 min)	Admin only
Guard police clearance	S3 Standard (SSE-KMS)	Pre-signed GET URL	300s	Admin only
User profile photo	S3 Standard	CloudFront public URL	1 day	Anyone
Booking receipt PDF	S3 Standard	Pre-signed GET URL	3600s (1 hour)	Booking user only
Analytics CSV export	S3 Standard	Pre-signed GET URL	900s (15 min)	Admin only
Payout report XLSX	S3 Standard	Pre-signed GET URL	900s	Admin only

```

# guards/serializers.py – serving photo via CloudFront (no
presigning needed)
class GuardDetailSerializer(serializers.ModelSerializer):
    profile_photo_url = serializers.SerializerMethodField()

    def get_profile_photo_url(self, obj) -> str | None:
        if not obj.profile_photo:
            return None

```

```

        # CloudFront public URL – no signing needed
        return f"https://cdn.bsecure.app/{obj.profile_photo.name}"

# guards/views.py – serving ID document via pre-signed URL
class GuardDocumentAccessView(APIView):
    permission_classes = [IsAuthenticated, IsAdminUser]

    def get(self, request, guard_id: int, doc_type: str):
        doc = get_object_or_404(GuardDocument, guard_id=guard_id,
                                document_type=doc_type)
        url = generate_presigned_get_url(s3_key=doc.s3_key,
                                         expiration_seconds=300)
        if not url:
            return Response({"detail": "Document unavailable."},
                            status=503)
        return Response({"url": url, "expires_in": 300})

```

7. CORS Configuration

```

[
  {
    "AllowedHeaders": [
      "Content-Type",
      "Authorization",
      "x-amz-date",
      "x-amz-content-sha256",
      "x-amz-security-token"
    ],
    "AllowedMethods": [
      "GET",
      "PUT",
      "POST",
      "HEAD"
    ],
    "AllowedOrigins": [
      "https://admin.bsecure.app",
      "https://bsecure.app",
      "http://localhost:3000",
      "http://localhost:8081",
      "exp://localhost:8081"
    ],
    "ExposeHeaders": [
      "ETag",
      "x-amz-request-id",
      "x-amz-id-2"
    ],
    "MaxAgeSeconds": 3600
  }
]

```

Apply with:

```
aws s3api put-bucket-cors \  
  --bucket bsecure-prod \  
  --cors-configuration file://cors.json
```

8. Lifecycle Rules

8.1 Lifecycle Rule JSON (AWS Format)

```
{  
  "Rules": [  
    {  
      "ID": "guard-documents-tiering",  
      "Status": "Enabled",  
      "Filter": {"Prefix": "media/guards/documents/"},  
      "Transitions": [  
        {  
          "Days": 90,  
          "StorageClass": "STANDARD_IA"  
        },  
        {  
          "Days": 365,  
          "StorageClass": "GLACIER"  
        }  
      ]  
    },  
    {  
      "ID": "guard-id-proof-tiering",  
      "Status": "Enabled",  
      "Filter": {"Prefix": "media/guards/id_proof/"},  
      "Transitions": [  
        {  
          "Days": 90,  
          "StorageClass": "STANDARD_IA"  
        },  
        {  
          "Days": 365,  
          "StorageClass": "GLACIER"  
        }  
      ]  
    },  
    {  
      "ID": "booking-receipts-compliance",  
      "Status": "Enabled",  
      "Filter": {"Prefix": "media/bookings/receipts/"},  
      "Transitions": [  
        {  
          "Days": 180,  

```

```

        "StorageClass": "STANDARD_IA"
    }
]
},
{
    "ID": "analytics-exports-expiry",
    "Status": "Enabled",
    "Filter": {"Prefix": "exports/analytics/"},
    "Expiration": {
        "Days": 90
    }
},
{
    "ID": "payout-reports-tiering",
    "Status": "Enabled",
    "Filter": {"Prefix": "exports/payouts/"},
    "Transitions": [
        {
            "Days": 180,
            "StorageClass": "STANDARD_IA"
        }
    ]
},
{
    "ID": "abort-incomplete-multipart-uploads",
    "Status": "Enabled",
    "Filter": {"Prefix": ""},
    "AbortIncompleteMultipartUpload": {
        "DaysAfterInitiation": 7
    }
},
{
    "ID": "expire-old-versions",
    "Status": "Enabled",
    "Filter": {"Prefix": ""},
    "NoncurrentVersionExpiration": {
        "NoncurrentDays": 90
    }
}
]
}

```

Apply with:

```

aws s3api put-bucket-lifecycle-configuration \
    --bucket bsecure-prod \
    --lifecycle-configuration file://lifecycle.json

```

9. Versioning & Backup

9.1 Enable Versioning

```
# Required for guard documents (immutable audit trail) and
# receipts (legal compliance)
aws s3api put-bucket-versioning \
    --bucket bsecure-prod \
    --versioning-configuration Status=Enabled
```

Versioning ensures that overwriting or deleting a guard's identity document creates a new version rather than destroying the original. The NoncurrentVersionExpiration lifecycle rule (90 days) prevents unlimited version accumulation for non-compliance files.

9.2 Cross-Region Replication (Disaster Recovery)

```
{
  "Role": "arn:aws:iam::123456789:role/bsecure-s3-replication",
  "Rules": [
    {
      "ID": "replicate-all-to-hyderabad",
      "Status": "Enabled",
      "Filter": {"Prefix": ""},
      "Destination": {
        "Bucket": "arn:aws:s3:::bsecure-prod-replica",
        "StorageClass": "STANDARD_IA",
        "ReplicationTime": {
          "Status": "Enabled",
          "Time": {"Minutes": 15}
        },
        "Metrics": {
          "Status": "Enabled",
          "EventThreshold": {"Minutes": 15}
        }
      },
      "DeleteMarkerReplication": {"Status": "Enabled"}
    }
  ]
}
```

```
aws s3api put-bucket-replication \
    --bucket bsecure-prod \
    --replication-configuration file://replication.json
```

9.3 S3 Object Lock for Receipts (Compliance Mode)

```
# Object Lock must be enabled at bucket creation time
aws s3api create-bucket \
```



```

--bucket bsecure-prod \
--region ap-south-1 \
--object-lock-enabled-for-bucket \
--create-bucket-configuration LocationConstraint=ap-south-1

# Set default retention: compliance mode, 7 years
aws s3api put-object-lock-configuration \
--bucket bsecure-prod \
--object-lock-configuration '{
  "ObjectLockEnabled": "Enabled",
  "Rule": {
    "DefaultRetention": {
      "Mode": "COMPLIANCE",
      "Years": 7
    }
  }
}'

```

Note: Object Lock is applied selectively via the media/bookings/receipts/ prefix only. Use tag-based object lock or apply in code when uploading receipts:

```

def upload_booking_receipt(booking_id: int, pdf_bytes: bytes):
    s3 = boto3.client("s3")
    key = f"media/bookings/receipts/{booking_id}/receipt.pdf"
    s3.put_object(
        Bucket=settings.AWS_STORAGE_BUCKET_NAME,
        Key=key,
        Body=pdf_bytes,
        ContentType="application/pdf",
        ObjectLockMode="COMPLIANCE",
        ObjectLockRetainUntilDate="2032-01-01T00:00:00Z", # 7
        years from 2025
    )
    return key

```

10. Security

10.1 Block Public Access (All Four Settings)

```

aws s3api put-public-access-block \
--bucket bsecure-prod \
--public-access-block-configuration '{
  "BlockPublicAcls": true,
  "IgnorePublicAcls": true,
  "BlockPublicPolicy": true,
  "RestrictPublicBuckets": true
}'

```

10.2 Bucket Policy — Deny HTTP, Allow CloudFront OAC

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyNonHTTPS",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3::bsecure-prod",
        "arn:aws:s3::bsecure-prod/*"
      ],
      "Condition": {
        "Bool": {
          "aws:SecureTransport": "false"
        }
      }
    },
    {
      "Sid": "AllowCloudFrontOAC",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3::bsecure-prod/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn":
            "arn:aws:cloudfront::123456789:distribution/
            E1ABCDEFGHIJKL"
        }
      }
    },
    {
      "Sid": "AllowAppRoleAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789:role/bsecure-ecs-task"
      },
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject"
      ],
      "Resource": "arn:aws:s3::bsecure-prod/media/*"
    }
  ]
}
```

10.3 Server-Side Encryption

```
# Default encryption: SSE-S3 (AES-256) for all objects
aws s3api put-bucket-encryption \
  --bucket bsecure-prod \
  --server-side-encryption-configuration '{
    "Rules": [{
      "ApplyServerSideEncryptionByDefault": {
        "SSEAlgorithm": "AES256"
      },
      "BucketKeyEnabled": true
    }]
  }'
```

Identity documents use SSE-KMS (set per-object in PrivateDocumentStorage — see section 3.3).

10.4 CloudTrail S3 Data Events

```
# Enable S3 data events in CloudTrail (audit every GetObject/
PutObject/DeleteObject)
aws cloudtrail put-event-selectors \
  --trail-name bsecure-prod-trail \
  --event-selectors '[{
    "ReadWriteType": "All",
    "IncludeManagementEvents": true,
    "DataResources": [{
      "Type": "AWS::S3::Object",
      "Values": ["arn:aws:s3:::bsecure-prod/"]
    }]
  }']
```

10.5 VPC Endpoint for S3

ECS tasks access S3 through a VPC Gateway Endpoint, keeping traffic off the public internet and reducing data transfer costs.

```
# Create VPC endpoint for S3 in the prod VPC
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-0abc12345def67890 \
  --service-name com.amazonaws.ap-south-1.s3 \
  --route-table-ids rtb-0abc12345
```

11. CloudFront Distribution

11.1 Distribution Configuration Summary

Setting	Value
Origin	S3 bucket bsecure-prod with Origin Access Control (OAC)
Custom domain	cdn.bsecure.app
ACM certificate	*.bsecure.app in us-east-1
Minimum TLS	TLS 1.2
HTTP to HTTPS	Redirect HTTP → HTTPS
Origin protocol	HTTPS only
Compress objects	Yes (gzip/brotli)

11.2 Cache Behaviours

Path Pattern	TTL	Query String Forwarding	Use
media/guards/photos/*	86400s (1 day)	None	Guard profile photos
media/users/photos/*	86400s	None	User profile photos
static/*	31536000s (1 year)	None	Static assets (immutable)
media/* (default)	86400s	None	All other media

```
# Invalidate CloudFront cache when a guard profile photo is
    updated
import boto3
from django.conf import settings

def invalidate_cloudfront(paths: list[str]):
    """
    paths: list of S3 key paths, e.g. ["media/guards/photos/42/
        profile.jpg"]
    """
    cf = boto3.client("cloudfront")
    cf.create_invalidation(
        DistributionId=settings.CLOUDFRONT_DISTRIBUTION_ID,
        InvalidationBatch={
            "Paths": {
```

```

        "Quantity": len(paths),
        "Items": [f"/{p}" for p in paths],
    },
    "CallerReference": str(uuid.uuid4()),
}
)

```

11.3 django-storages Auto-Invalidation on File Save

```

# config/storages.py – extend MediaStorage to auto-invalidate on
# save
from django.db.models.signals import post_save
from django.dispatch import receiver
from guards.models import Guard

@receiver(post_save, sender=Guard)
def invalidate_guard_photo_cache(sender, instance, **kwargs):
    if instance.profile_photo:
        try:
            invalidate_cloudfront([instance.profile_photo.name])
        except Exception as e:
            logger.warning("CloudFront invalidation failed for
guard %s: %s", instance.id, e)

```

12. Cost Optimisation

12.1 Storage Class Comparison

Storage Class	Min Duration	Retrieval Fee	Monthly (per GB)	Best For
Standard	None	None	~\$0.023	Frequently accessed (profile photos)
Standard-IA	30 days	Per GB	~\$0.0125	Infrequently accessed (old receipts)
Glacier Instant	90 days	Per GB	~\$0.004	Archives with millisecond retrieval
Glacier Flexible	90 days	Per GB (3–5h)	~\$0.0036	Deep archive,

Storage Class	Min Duration	Retrieval Fee	Monthly (per GB)	Best For
Intelligent-Tiering	None	Monitoring fee	Variable	rarely retrieved Unknown access patterns

12.2 S3 Intelligent-Tiering for media/

Enable after 6 months when access patterns are understood:

```
aws s3api put-bucket-intelligent-tiering-configuration \
  --bucket bsecure-prod \
  --id media-intelligent-tiering \
  --intelligent-tiering-configuration '{
    "Id": "media-intelligent-tiering",
    "Status": "Enabled",
    "Filter": {"Prefix": "media/"},
    "Tierings": [
      {"Days": 90, "AccessTier": "ARCHIVE_ACCESS"},
      {"Days": 180, "AccessTier": "DEEP_ARCHIVE_ACCESS"}
    ]
  }'
```

12.3 Estimated Monthly Cost — Year 1

Assumptions: 50 GB media, 10 GB exports, 1M S3 GET requests/month, CloudFront 100 GB/month transfer.

Component	Size / Requests	Unit Price	Monthly Cost
S3 Standard storage	50 GB	\$0.023/GB	\$1.15
S3 Standard-IA (old docs)	10 GB	\$0.0125/GB	\$0.13
S3 PUT requests	50,000	\$0.005/1K	\$0.25
S3 GET requests	1,000,000	\$0.0004/1K	\$0.40
CloudFront data transfer	100 GB	\$0.085/GB (India)	\$8.50
CloudFront HTTPS requests	2,000,000	\$0.0075/10K	\$1.50
KMS API calls (SSE-KMS)	10,000	\$0.03/10K	\$0.03
S3 replication transfer	5 GB	\$0.02/GB	\$0.10

Component	Size / Requests	Unit Price	Monthly Cost
Total estimate			~\$12/month

Year 1 costs remain low. The dominant cost driver beyond 100 GB will be CloudFront transfer. Enable S3 Intelligent-Tiering after 6 months to automatically move infrequently accessed guard documents to cheaper tiers.

12.4 Cost Monitoring

Set a billing alert for S3 + CloudFront

```
aws cloudwatch put-metric-alarm \
  --alarm-name "bsecure-s3-cost-alert" \
  --alarm-description "S3 + CloudFront cost exceeds $50/month" \
  --metric-name EstimatedCharges \
  --namespace AWS/Billing \
  --statistic Maximum \
  --period 86400 \
  --threshold 50 \
  --comparison-operator GreaterThanThreshold \
  --dimensions Name=ServiceName,Value=AmazonS3 \
  --alarm-actions arn:aws:sns:us-east-1:123456789:bsecure-
    billing-alerts
```